qvac-swift: A Native Swift Client for On-Device AI Inference

Kenny (frankolien)
gkenny896@gmail.com · github.com/frankolien/qvac-swift

July 2026 · v0.1.0

Abstract. A purely native Swift client would allow applications on Apple platforms to run large-language-model inference on-device without carrying a JavaScript application layer along for the ride. QVAC, Tether's local-first AI platform, already runs its inference engines in a separate worker process and reaches them over a compact binary RPC protocol — which means the client half of the conversation is only a protocol away from any language. We implement that protocol natively: a frame codec verified byte-for-byte against the reference encoder, a session layer that multiplexes concurrent calls and converts transport death into immediate typed failure, flow control that lets a slow reader pause a fast model, and a 37-method typed API generated from the platform's own machine-readable contract. The implementation is verified bottom-up against the real system rather than its documentation, a process that surfaced five places where the two disagree. The result runs end-to-end against the shipping worker: a 773 MB model fetched with streamed download progress, then a completion streamed token-by-token — 45 tokens in 825 ms, entirely on one machine, with nothing leaving the device.

1. Introduction

Running AI models on the device that uses them is attractive for reasons that get stronger every year: nothing sensitive leaves the machine, there is no per-token bill, and there is no network between a prompt and its answer. QVAC [1] packages this idea as a platform — LLM completion, speech, retrieval, translation, and image generation, all executing locally — but its official SDK is JavaScript. A Swift developer who wanted QVAC in a Mac or iPhone app had to embed a JavaScript application layer just to reach an engine that was already native underneath.

The observation this project is built on is architectural: QVAC's SDK does not *contain* the AI. Inference runs in a separate worker process on the Bare runtime [3], and the SDK client is an RPC consumer talking to it over a socket. The JavaScript is not load-bearing — the protocol is. Any implementation that produces the same bytes is a first-class client.

This paper describes that protocol and the engineering of its Swift implementation, from the individual bytes on the wire up to a typed `async/await` API, in enough detail that a reader could rebuild it. Each section opens with a plain-language statement of the problem before the detail, because the ideas are simple even where the encoding is fiddly.

2. System Model

> **Plain terms** — *your app and the AI engine are two separate programs on the same machine, talking through a private pipe. The surprise is who calls whom: your app opens the pipe and listens; the engine dials in.*

Two processes cooperate. The *client* is the application — here, Swift code. The *worker* is a Bare-runtime process hosting the inference engines (llama.cpp and friends) behind a dispatch loop. They communicate over a Unix domain socket or a local TCP connection carrying the `bare-rpc` [2] protocol described in §3.

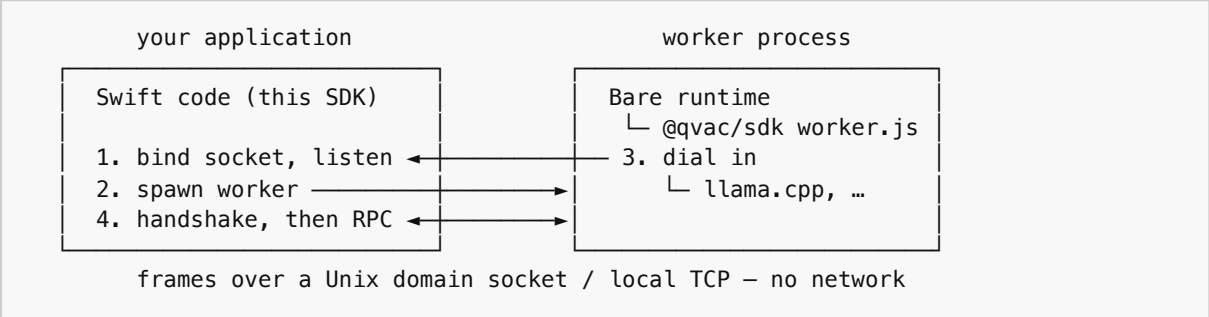

Figure 1. Process model. The client listens; the worker dials in — the reverse of the intuitive direction.

The connection direction is the first thing the source code corrects about the written documentation: the client does not connect to a serving worker. The client binds an endpoint, spawns the worker with that endpoint in its argument vector, and accepts the worker's dial-in:

```
bare worker.js '{"QVAC_IPC_SOCKET_PATH": "/tmp/qvac.sock", "HOME_DIR": "…"}'
```

This ordering matters because whoever spawns the worker must have already bound the socket, or the worker's dial-in races against the listener. On desktop, one call (`launchWorker`) performs the whole sequence: bind, spawn, accept, handshake. On iOS, where an app may not spawn processes at all, the worker runs instead as a Bare *worklet* embedded in the app, and the same bytes flow over the worklet's IPC pipe — the transport is a three-requirement seam precisely so that the entire protocol stack above it is identical on both platforms.

3. The Wire Format

> **Plain terms** — *a socket is a hose of bytes with no boundaries. The protocol first cuts the hose into frames (each one announces its own length), then packs numbers inside each frame using as few bytes as possible.*

3.1 Frames

TCP and Unix sockets deliver a byte stream, not messages: one logical message may arrive split across many reads, or several may arrive glued together in one. Every `bare-rpc` message therefore begins with a 4-byte little-endian unsigned length counting the body that follows. The receiver accumulates bytes until a full body is present, decodes it, and repeats. Our reassembler is amortized O(n) in total bytes received and is tested at the pathological extreme — the entire conversation delivered one byte at a time.

3.2 Variable-length integers

Inside a frame, integers use Bitcoin-style *CompactSize* encoding: small numbers — by far the most common — cost one byte, and three escape values announce wider encodings:

Value range	Encoding	Size
0 … 252	the byte itself	1
253 … 65,535	`0xFD` + uint16 LE	3
65,536 … $2^{32}-1$	`0xFE` + uint32 LE	5
2^{32} … $2^{64}-1$	`0xFF` + uint64 LE	9

Strings and byte blobs are a CompactSize length followed by the raw bytes. The fixture suite (§9) deliberately includes payloads that straddle each width boundary, because off-by-one errors at 252/253 and 65,535/65,536 are exactly where hand-rolled codecs break.

3.3 Signed integers: zigzag

> **Plain terms** — *the varint scheme above only knows non-negative numbers, so signed values are first folded: count 0, −1, 1, −2, 2, … and give each its position. Small negatives stay small instead of becoming huge.*

A two's-complement −1 is all ones — as an unsigned 64-bit number, 18,446,744,073,709,551,615, the worst possible varint. Zigzag encoding maps signed to unsigned by interleaving: `encoded = (n << 1) ^ (n >> 63)`, so values near zero stay near zero in either sign:

n	0	−1	1	−2	2	−3	52,401
encoded	0	1	2	3	4	5	104,802

The protocol uses this for error numbers: the POSIX errno −3 travels as 5, and QVAC's error code 52401 travels as 104,802. Both appear verbatim in the byte fixtures.

3.4 Message anatomy

There are exactly three message types — `REQUEST` = 1, `RESPONSE` = 2, `STREAM` = 3. A request body is five fields, all CompactSize: the type, an id that pairs the eventual response to its request, a command number (§7), a flags word, and the JSON payload as a length-prefixed blob. Figure 2 shows a complete 21-byte request as it appears on the wire:

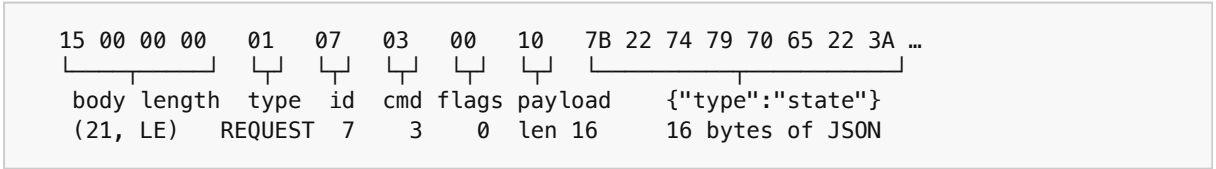

Figure 2. A complete unary request frame: 4-byte length prefix, then five CompactSize-encoded fields.

Payloads themselves are JSON — the protocol is a binary envelope around text records, taking framing and flow control from the binary layer and schema evolution from JSON. Streamed *re-*

sponses pack multiple records per frame as newline-delimited JSON; streamed *requests* are parsed one whole record per frame — an asymmetry visible only in the server's source, and the kind of detail that separates a working client from a plausible one.

`STREAM` messages carry no command; they carry a bitmask of flags that compose into the stream lifecycle:

Flag	Bit	Meaning
`OPEN`	`0x001`	stream exists; acknowledges the opener
`CLOSE`	`0x002`	graceful end (with `ERROR`: failure)
`PAUSE` / `RESUME`	`0x004 / 0x008`	flow control (§5)
`DATA`	`0x010`	frame carries payload records
`END`	`0x020`	half-close: no more data this direction
`DESTROY`	`0x040`	abandon the stream (consumer cancelled)
`ERROR`	`0x080`	frame carries an error struct
`REQUEST` / `RESPONSE`	`0x100 / 0x200`	which half of the call this stream is

4. Multiplexing and the Life Signal

> **Plain terms** — *many calls share one pipe, told apart by id numbers. And the deadliest failure isn't an error message — it's silence. If the engine dies, every call waiting on it must fail immediately, not hang forever.*

All calls share one connection. Each outgoing request takes the next id; a table maps ids to suspended Swift continuations; when a response arrives, the dispatcher looks up its id and resumes exactly that caller. The table lives inside a Swift `actor`, so the compiler — not convention — guarantees no interleaving corrupts it. Requests with id 0 are fire-and-forget events that expect no response.

The invariant we consider the heart of the session layer is the *life signal*: a call may succeed or fail, but it may never hang. The reference JavaScript client documents the hazard — if the worker dies while requests are in flight, promises with no process behind them simply never settle. Our session fuses the transport to the request table: the moment the read loop observes transport death (socket EOF, reset, or worker crash), it tears down every pending continuation and every open stream with a typed `channelClosed` error, at that instant. The crash test in the live suite kills a real worker mid-call and asserts that all in-flight calls fail promptly rather than time out.

Cancellation composes with the same machinery in the opposite direction: cancelling the Swift `Task` consuming a stream sends `DESTROY` to the worker, so a model stops generating tokens nobody is reading. Structured concurrency on the client maps onto stream lifecycle on the wire.

5. Backpressure

> **Plain terms** — *if the engine produces faster than your code consumes, the excess has to go somewhere. Instead of buffering without limit, the client tells the engine "pause" when its bucket is nearly full and "resume" once it drains — like a float valve in a water tank.*

Token generation and download progress are producer-driven, and an unbounded buffer is a slow-motion memory leak. Streams here are pull-based: records buffer only until the consumer's `AsyncSequence` asks for the next one. The buffer is bounded by hysteresis — when buffered bytes reach the high-water mark (1 MiB), the session sends `PAUSE`; when the consumer drains it to the low-water mark (256 KiB), it sends `RESUME`:

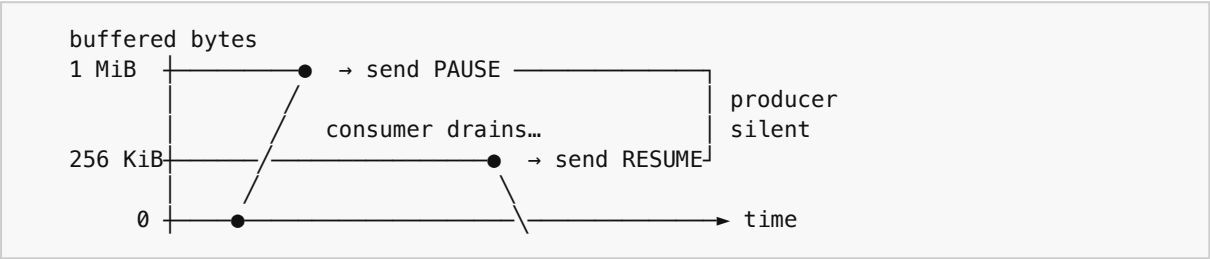

Figure 3. Two thresholds instead of one prevent flapping — the same reason a thermostat has a dead band.

A single threshold would oscillate — pause, drain one record, resume, refill, pause — flooding the wire with control frames. The gap between the marks amortizes each control frame over hundreds of kilobytes of payload. The same protocol works in reverse on duplex calls: the worker may `PAUSE` the client's outbound record stream, and the session suspends writers (as awaiting Swift callers, not blocked threads) until `RESUME`.

6. Call Shapes

Every one of QVAC's methods is one of three shapes, and one runtime twist:

Unary — one JSON request, one JSON response. Configuration, model management, registry queries.

Server-stream — one request, a stream of typed records back. Token completion is the canonical case: each record carries content deltas that the application prints as they arrive.

Duplex — typed records flowing both directions concurrently, with independent flow control and half-close: live transcription sends audio chunks up while transcript segments stream down, then `END` says "no more audio" while results continue. The server source holds a subtlety here: the frame that opens a duplex call carries *no payload* — the request object travels as the first record on the request stream. A client that packs the request into the opener (the natural guess) desynchronizes the server's record parser on the first call.

Progress promotion — four methods (`loadModel`, `downloadAsset`, `rag`, `finetune`) change shape at runtime: with `withProgress` set (for some, gated further on the operation), a unary call becomes a stream of progress records ending in the final result, partitioned by each record's `type` tag. The conditions are JavaScript expressions in the platform's manifest; the Swift predicates are hand-written and a test asserts them verbatim against the manifest, so a contract change flips the

test before it flips a user. The unary overloads *refuse* promotable requests with a thrown error — making a would-be silent hang into an immediate, explanatory failure.

7. Method Dispatch: the Command Counter

> **Plain terms** — *we assumed each API method had its own wire number, like ports. Wrong: the number is just a call counter, and the method's real name rides inside the JSON. Getting this wrong produces a client that works exactly once.*

The request frame carries a `command` integer, and every RPC design pattern suggests it identifies the method. Reading the worker's dispatch source falsifies this: `command` is a per-call counter — 1, 2, 3, … — used only to key streams and responses, and the method is named by the `type` field inside the JSON payload. The handshake (`__init_config`) is command 1 not because 1 means "initialize" but because it is always the first call any client makes.

This is the most instructive of the five documentation corrections (Table 1, §9): a client built on the fixed-number assumption completes the handshake perfectly — one call, command 1, works — and fails mysteriously on the second call ever made. It is also why this implementation was built from source rather than from the brief.

8. The Generated Surface

> **Plain terms** — *the 37 API methods and their types aren't hand-written; a generator reads QVAC's own machine-readable API description and writes the Swift. When QVAC changes, we re-run the generator instead of hunting for what moved.*

QVAC publishes its API as data: a JSON Schema of every request and response, a manifest of methods and call shapes, and an error taxonomy. We vendor these files pinned to an exact upstream commit (`tetherto/qvac@fadfaedc`) and generate the entire typed surface from them: 37 methods across all three call shapes, 13,642 lines of model types, and 136 named error codes.

The generator's job is mostly turning JSON Schema's loose unions into Swift's strict enums: discriminated unions become enums whose decoder peeks the discriminator field before choosing an arm; unions of primitive constants collapse to the primitive; integer enums take their case names from the schema's varname annotations. Two properties are enforced rather than hoped for: output is deterministic (same contract in, byte-identical Swift out), and name collisions are hard errors — never auto-suffixed counters, which would let two different types silently swap meanings between regenerations. CI runs the generator in `--check` mode on every commit, so committed output that drifts from the contract fails the build.

The alternative — a hand-written API surface — is how client libraries rot: upstream adds a field, the client silently drops it. Here the contract is the single source of truth, and staleness is a build failure instead of a latent bug.

9. Verification

> **Plain terms** — *every layer is tested against the real thing, not against our own understanding of it. A codec tested only against itself can be wrong twice in matching ways and still pass.*

The implementation was verified bottom-up, each rung of the ladder trusting the one below:

```
5 ── real inference        773 MB model download with live progress,
│                          token-streamed completion, clean shutdown
4 ── shipping worker       the actual @qvac/sdk worker under the real
│                          bare runtime: handshake, state, registry
3 ── live protocol peer     a Node process running real bare-rpc dials
│                          into the Swift listener: unary, streams,
│                          duplex, crash teardown — TCP and Unix
2 ── session invariants     scripted transports: life signal, races,
│                          END with buffered records, backpressure
1 ── byte fixtures          85 wire vectors generated by the REFERENCE
                           JS encoder, reproduced byte-for-byte
```

Figure 4. The verification ladder. Fixtures come from the reference implementation, never from this one.

The base rung is the load-bearing one. The 85 fixture vectors are produced by the reference JavaScript encoder and cover every varint width boundary, the signed errno range through zigzag, and payload sizes across each length-prefix transition; the Swift codec must reproduce them byte-for-byte and re-parse them from one-byte chunks. At the top, the ladder ends with the system doing its actual job: §10's inference run used the promoted `loadModel` shape, real registry download with resume-after-timeout, and a token stream through the generated types — every layer of this paper exercised at once, against software we did not write.

Building from source rather than documentation surfaced five places where they disagree; a client written from the brief alone would embody all five:

	The reasonable assumption	**What the source does**
1	Client connects to a listening worker	Client listens; worker dials in (§2)
2	`command` identifies the method	Per-call counter; method named in payload (§7)
3	Duplex opener carries the request	Opener empty; request is first stream record (§6)
4	Generate types from TypeScript definitions	Contract is JSON: schema + manifest + error codes (§8)
5	iOS spawns the worker like desktop	iOS cannot spawn; worker runs as an embedded worklet (§2)

Table 1. Documentation vs. source. Each cost a rewrite avoided by reading first.

In total: 52 tests, run on every commit on both Linux and macOS, plus the codegen staleness gate. The wire and session layers have zero dependencies and never touch a Bare binary, which is why the full protocol suite runs on a bare Linux container in seconds.

10. Performance

Measured on the development machine (Apple-silicon MacBook), driving the shipping worker through this SDK with Llama-3.2-1B-Instruct (Q4_0 quantization, 773 MB):

Operation	Measured
Registry query, decoded into generated types	724 models, sub-second
First model fetch (streamed progress, resumable)	773 MB; survived a mid-download timeout and resumed from the partial blob store
Model load from local cache	≈ 15 s
Completion, first run	45 tokens in 825 ms (≈ 55 tok/s)
Completion, subsequent runs	38–47 tokens in 524–622 ms (≈ 70–75 tok/s)

The generation numbers are the engine's — the point of this SDK is to add approximately nothing to them. Per token, the client's work is decoding a JSON record from an already-reassembled frame and resuming one continuation; the wire layer's reassembly is amortized O(n) in bytes received, and backpressure control frames are amortized over the 768 KiB hysteresis band. An interactive multi-turn chat loop (history accumulated client-side and resent per turn, the same mechanism every chat assistant uses) runs comfortably within these figures.

11. Conclusion

We began with the observation that QVAC's client is only a protocol away from any language, and ended with a Swift implementation whose every layer is verified against the real system: reference bytes at the bottom, the shipping worker at the top, and a model answering questions on-device through the middle. The design reduces to a few commitments — frames over a hose of bytes, ids over shared pipes, a life signal instead of hangs, hysteresis instead of unbounded buffers, generated types instead of hand-written rot, and fixtures from the reference instead of from ourselves. None of these ideas is novel; the work was applying all of them, and trusting source code over documentation at the five points where they part ways. What remains — richer platform bindings, more engines, more devices — is application, not architecture: the protocol floor this paper describes is the part that had to be right, and it is the part that is now proven.

References

[1] Tether, "QVAC: a local-first AI platform," github.com/tetherto/qvac. Contract pinned at commit `fad-faedc`.

[2] Holepunch, "bare-rpc: lightweight RPC over any duplex stream," github.com/holepunchto/bare-rpc.

[3] Holepunch, "Bare: a lightweight JavaScript runtime," github.com/holepunchto/bare.

[4] "@qvac/sdk," npm registry, v0.16.0 — provides the worker bundle verified against in §9–10.

[5] G. Gerganov et al., "llama.cpp: LLM inference in C/C++," github.com/ggml-org/llama.cpp.

[6] Meta AI, "Llama 3.2 1B Instruct," GGUF Q4_0 quantization via unsloth, huggingface.co.

[7] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — origin of the CompactSize varint encoding in §3.2.

[8] frankolien, "qvac-swift," github.com/frankolien/qvac-swift — the implementation, tests, and fixtures described here.
